Code Artifacts — Vehicle Washing Management System

Cover Page

Table 1

Document Identification Data

Field	Value
Project	Vehicle Washing Management System (Vehicle Washing / LavaRápido)
Deliverable	Code artifacts (section 2.5.4)
Program	Software Analysis and Development Technologist (ADSO)
Group (ficha)	3145556
Apprentices	Brayan Estiven Patiño Cabrera, Juan Camilo Guilombo Osorio, Merari Urbano Quintero
Instructor	Karol Daniela Correa
Training center	Industria
Date	November 4, 2026

The system's code lives in 10 GitHub repositories: the web application, the mobile application, and 8 backend repositories. Each one has a GitHub Actions workflow (`.github/workflows/ci.yml`) that builds it and runs its tests on every change (in `lavarapido-infra` it validates the deploy scripts, the Docker Compose files and the end-to-end test syntax); `lavarapido-infra` also has `deploy.yml`, the workflow that updates the server. Since October 26, 2026 continuous deployment runs on its own: the "Pedir despliegue" job in the `ci.yml` of 8 repositories launches that workflow with every merge into `main`, and `cd.yml` in the mobile app publishes the over-the-air update if the native fingerprint matches the published APK's (*versionCode* 6). On October 23, 2026 `v1.0.0` was tagged in 8 of the 10 repositories and on the 24th in payment-service; on October 26 `v1.1.0` shipped in all 10 (the mobile app's first) and `v1.1.1` in the web, the mobile app, payment-service, and infra, and on October 27 `v1.1.1` in customer-service and booking-service and `v1.1.2` in the web, payment-service, infra, and the mobile app (the latter only with a corrected text, which reaches the phones over the internet). That same October 27, payment-service `v1.1.3` was merged into `main`, with no passwords written in the code (the service does not start without `DB_PASSWORD` and requires the credentials

of a remote RabbitMQ); its tag and release were published on November 4, 2026. `main` has been protected in the 10 repositories since `v1.0.0` (it only takes changes through a pull request with one approval and a green CI) and the public server runs `main`, which continuous deployment updates on every merge. The published web image runs nginx without `root` on port 8080. This document inventories the two clients and explains how to build and run them; the backend is described only as far as the clients need it. Source code is not reproduced.

1. Repository Inventory

Table 2

System Repositories (`develop` Branch, as of October 24, 2026)

Artifact	Repository	Technology	Commits	Last commit
Web application	Front-end-proyecto-web	Angular 21.2.25, TypeScript 5.9	576	2026-10-24
Mobile application	LavadoVehicular-Movil	React Native 0.86.3, Expo ~57, TypeScript 6.0	296	2026-10-24
Infrastructure	lavarapido-infra	Docker Compose	151	2026-10-24
API Gateway	lavarapido-api-gateway	Spring Boot 4.0.8, Java 21	42	2026-10-24
security-service	lavarapido-security-service	Spring Boot 4.0.8, Java 21	100	2026-10-24
customer-service	lavarapido-customer-service	Spring Boot 4.0.8, Java 21	41	2026-10-24
booking-service	lavarapido-booking-service	Spring Boot 4.0.8, Java 21	102	2026-10-24
operations-service	lavarapido-operation-service	Spring Boot 4.0.8, Java 21	93	2026-10-24
payment-service	lavarapido-payment-service	ASP.NET Core 8	85	2026-10-24
notification-service	lavarapido-notification-service	Spring Boot 4.0.8, Java 21	65	2026-10-24

> *Note.* The commits and dates are those of the October 24, 2026 cut-off and were not counted again. The versions in force on `main` are those of October 27, 2026: `v1.1.0` in the API Gateway, security-service, operations-service, and notification-service; `v1.1.1` in customer-service and booking-service; `v1.1.2` in the web, the mobile app, and infra; and `v1.1.3` in payment-service (merged into `main` on October 27 and tagged on November 4, 2026).

Table 3

Client Size

Artifact	Source files (`.ts`, `.tsx`, `.html`, `.scss`)	Lines	Screens
Web (`Web/src`)	449	48,202	35 routes
Mobile (`LavadoVehicular-Movil/src`)	296	35,540	31 screens

> *Note.* Count of October 21, 2026, with `git ls-files` and `wc -l` over `src`; includes tests and comments.

2. Web Application

The web client is an Angular 21 SPA (Angular, n.d.) with standalone components, one lazy-loaded module per role, and translations into four languages (es, en, fr, pt). Every call goes to `/api/v1`, which the development server proxies to the gateway at `localhost:8080`.

2.1 Structure

Table 4

Folders in `Web/src/app`

Folder	Contents
`core/constants`	`API_BASE_URL = '/api/v1'` (`core/constants/api.ts:4`)
`core/guards`	`authGuard` (session and role) and `guestGuard` (hides login from users who already have a session)
`core/interceptors`	`authInterceptor`: adds the JWT and ends the session on a 401 while the session is still open (it ignores a late 401 after logging out)
`core/services`	17 HTTP and state services: `auth`, `booking-api`, `vehicles`, `operations-api`, `payments-api`, `promotions-api`, `loyalty-api`, `notifications`, `preferences`, `role-permissions-api`, `user-admin`, `user-session`, `inspection-api`, `inspection-photos`, `local-storage`, `notification-watcher`, `api`; `auth` also follows sign-in and sign-out in the other tabs of the same browser
`core/models`, `core/utils`	DTO interfaces and utilities (for example, API error translation and `image-compress.ts`, which shrinks the receipt image to at most 2 MB before uploading it)
`features/landing`	Public page: navbar, hero, about us, features, services, download, contact, frequently asked questions, and footer
`features/auth`	Login, sign-up, and three-step password recovery
`features/client`	8 client pages + `viewmodels/client.viewmodel.ts` (layout)
`features/operator`	8 operator pages
`features/admin`	13 administrator pages + 5 stores (`reservations`, `payments`, `operators`, `schedule`, `business`)
`features/not-found`	Address not found page (route `**`)
`features/public-report`	Read-only inspection report (`/report/:token`)
`shared/components`, `shared/dialogs`, `shared/pipes`	12 reusable components and 20 dialogs (confirm, assign operator, rate, review payment, export…)
`assets/i18n`	`es.json`, `en.json`, `fr.json`, `pt.json`

2.2 Main Dependencies

Table 5

Web Libraries (`Web/package.json`)

Library	Version	Purpose
`@angular/*` (core, router, forms, material, cdk)	21.2.25 (Material and CDK 21.2.14)	Framework and UI components
`@ngx-translate/core`, `http-loader`	^17.0.0	Translations
`chart.js`	^4.5.1	Dashboard and report charts
`jspdf`, `jspdf-autotable`	^4.2.1, ^5.0.8	Export reports to PDF
`xlsx`, `file-saver`	0.20.3 (SheetJS CDN), ^2.0.5	Export to Excel and download files
`iconify-icon`	^3.0.2	Icons
`vitest`, `jsdom` (development)	^4.1.7, ^28.0.0	Unit tests with `@angular/build:unit-test`

2.3 Build and Run

Requirements: Node.js 22 LTS (the version of the CI and of the web Docker image, `node:22-alpine`), and npm 11 (`packageManager: npm@11.6.2`). Screens load data only when the backend is running (section 4).

```
cd Web
npm install          # install dependencies
npm start            # ng serve at http://localhost:4200, proxying /api ->
http://localhost:8080
npm run build        # production build in dist/web/browser
npm test             # unit tests (Vitest)
```

The proxy lives in `Web/proxy.conf.json` and is enabled in `angular.json` (`proxyConfig`). The production build has size budgets: it fails if the initial bundle exceeds 1 MB or a component style exceeds 16 kB (`angular.json`, `budgets`). In production, `dist/web/browser` can be served by any static server that forwards `/api` to the gateway.

According to SRS v7 (RNF-008), the web client must work on current versions of Chrome, Firefox, and Edge and be usable from 360 px wide.

3. Mobile Application

The mobile app is built with Expo 57 on React Native 0.86 (Expo, n.d.; React Native, n.d.) and targets Android 8.0 or later (RNF-008 in SRS v6). The web build (`react-native-web`) is used only for development and testing. It talks only to the gateway and discovers the IP of the computer running Expo by itself, so no setup is needed on a local network.

3.1 Structure

Table 6

Folders in `LavadoVehicular-Movil/src`

Folder	Contents
`app/config`	i18next with `es`, `en`, `fr`, `pt` (`app/config/locales`)
`app/providers`	`AppProviders`, sync of the account preferences and the push token, and `AppUpdatesCheck` (if the server published a newer APK, it must be installed before going on)
`app/theme`	Theme context, colors, and typography
`core/api`	`config.ts` (gateway URL and `WEB_URL` for public links), `httpClient.ts` (fetch with JWT, JSON or *multipart*, and a 15 s timeout or the one each call asks for), `apiError.ts`
`core/navigation`	`RootNavigator` (screens per role) and route types
`core/services`	`auth`, `booking`, `vehicles`, `operations`, `payments` (with `receiptImage.ts`, which compresses the receipt to at most 2 MB), `notifications`, `users`, `appUpdates` (reads `/app/version.json`)
`core/storage`	Session in `expo-secure-store` (AsyncStorage on web)
`modules/auth`, `client`, `operator`, `admin`	Screens, components, models, hooks, and view models for each role
`shared`	Per-role layouts, shared views (profile, notifications, settings), validators, components, and utilities

3.2 Main Dependencies

Table 7

Mobile Libraries (`LavadoVehicular-Movil/package.json`)

Library	Version	Purpose
`expo`	~57.0.30	Build and runtime platform
`react`, `react-native`	19.2.3, 0.86.3	User interface
`@react-navigation/native`, `native-stack`	^7.3.13, ^7.18.5	Navigation
`expo-secure-store`, `@react-native-async-storage/async-storage`	~57.0.4, 2.2.0	Session storage
`i18next`, `react-i18next`	^26.4.1, ^17.0.13	Translations
`expo-print`, `expo-sharing`, `expo-file-system`, `expo-document-picker`	~57.0.x	Export reports and attach receipts
`expo-image-picker`, `expo-image-manipulator`	~57.0.20, ~57.0.21	Take or pick the inspection report photos and shrink them before uploading
`expo-notifications`, `expo-updates`	~57.0.23, ~57.0.26	Push notifications and over-the-air (OTA) updates
`react-native-svg`, `react-native-reanimated`	15.15.4, 4.5.1	Charts and animations

3.3 Configuration

The backend URL is chosen in this order (`src/core/api/config.ts`):

1. `EXPO_PUBLIC_API_URL` in `.env.local`, if defined.
2. The IP of the computer running Expo (the same one the phone used to open the app).
3. Per platform: `http://10.0.2.2:8080/api/v1` on the Android emulator and `http://localhost:8080/api/v1` on the web build.

The `.env.example` template explains the optional variables; `.env.local` is not committed. The phone and the computer must be on the same Wi-Fi network, and the firewall must allow port 8080.

3.4 Build and Run

```
cd LavadoVehicular-Movil
npm install
npm start                # expo start: scan the QR code with Expo Go
npm run android          # open on an Android emulator or device
npm run web              # web version at http://localhost:8081
npx expo start --clear   # restart and clear the cache after changing .env.local
```

For development the app runs through Expo Go. The APK is built with EAS (`eas.json`, `preview` and `production` profiles), and the `npm run publish:apk` and `npm run publish:update` scripts publish a new APK on the server or an over-the-air update (see the Deployment Document, folder 07).

4. Backend Required by the Clients

The web and mobile clients only need the gateway at `localhost:8080`; behind it, the six services, SQL Server, and (if events are enabled) RabbitMQ must be running. Everything starts from `Backend/lavarapido-infra` with Docker.

```
cd Backend/lavarapido-infra
cp .env.example .env                          # fill in secrets; .env is never
committed
docker compose up -d                          # only SQL Server, Mailpit, and RabbitMQ
(services run from the IDE)
docker compose --profile app up -d --build    # the whole backend in containers
```

Table 8

Variables in `.env.example` (No Values)

Group	Variables
Database	`DB_PASSWORD` (required), `DB_NAME`, `DB_USERNAME`, `DB_URL`
Security	`JWT_SECRET` (required), `INTERNAL_API_KEY` (calls between services and automatic assignment), `CORS_ALLOWED_ORIGINS`
Email	`MAIL_ENABLED`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`
Messaging	`MESSAGING_ENABLED`, `RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_USERNAME`, `RABBITMQ_PASSWORD`; `AUTO_ASSIGN_ENABLED=false` (optional) turns off only the automatic assignment
Push	`PUSH_ENABLED` (and `EXPO_ACCESS_TOKEN` in the compose file)
Payment at booking (booking-service)	`BOOKING_CASH_ACTIVE_LIMIT` (active cash bookings per customer, 1), `BOOKING_PAYMENT_WINDOW` (deadline to upload the receipt of a digital booking, 30m), `BOOKING_PAYMENT_EXPIRY_ENABLED` and `BOOKING_PAYMENT_EXPIRY_INTERVAL` (cancellation with `PAYMENT_TIMEOUT` of overdue digital bookings: on and every 60s); commented out in `.env.example`, with defaults

Note. The variables of October 21, 2026 have defaults, are commented out in `.env.example`, and the compose file passes them to the containers since October 22, 2026: `SESSION_CHECK_ENABLED`, `SESSION_CHECK_CACHE_TTL`, and `SESSION_CHECK_TIMEOUT` (gateway session check: on, 30 s, and 3 s); `BOOKING_EXPIRY_CHECK_ENABLED` and `BOOKING_EXPIRY_CHECK_INTERVAL` (cancellation of scheduled bookings that expired without confirmation, in booking-service: on and every 5 minutes); `BOOKING_EXPIRY_GRACE_MINUTES` (60-minute grace after the end of the booking, used by booking-service and operations-service); and `FOLLOW_UP_ENABLED`, `FOLLOW_UP_INTERVAL`,

> and `FOLLOW_UP_LOOK_BACK_DAYS` (follow-up of the bookings that ended and nobody closed, in operations-service: on, every 5 minutes, and looking back 28 days; it needs `INTERNAL_API_KEY`). They are changed in `.env`. Since October 23, 2026 there are also `ALLOWED_EMAIL_DOMAINS` (email domains, `gmail.com` by default) and `DEMO_ADMIN_PASSWORD`, `DEMO_OPERATOR_PASSWORD`, and `DEMO_CLIENT_PASSWORD` (demo accounts of the `dev` profile; empty, the account is not created). Since October 24, 2026 customer-service receives `BOOKING_SERVICE_URL` (`http://booking-service:3003`) and `INTERNAL_API_KEY`; the gateway has `AUTH_RATE_LIMIT_*`, `TRUSTED_PROXIES`, and `ROUTE_CONNECT_TIMEOUT`/`ROUTE_READ_TIMEOUT`, and operations-service `START_LATE_LIMIT` (4 h), with default values and not yet in the compose file; and `BOOKING_EXPIRY_CHECK_*` no longer have any effect.

To run a Java service from the console, use its Maven Wrapper (`./mvnw spring-boot:run` inside the service folder) with Java 21 (Spring, n.d.). payment-service runs with `dotnet run --project src/PaymentService.Api` and requires the .NET 8 SDK (Microsoft, n.d.). When starting with `--profile app`, the folders must be named after their repositories (`lavarapido-security-service`, `lavarapido-operation-service`), because that is how the compose file references them.

Table 9

Ports to Check That Everything Is Up

Service	Health check URL
API Gateway	`http://localhost:8080/actuator/health`
Java services	`http://localhost:300X/actuator/health` (X = 1, 2, 3, 4, 6) and Swagger at `/swagger-ui.html`
payment-service	`http://localhost:3005/actuator/health` and `/health` (both check SQL Server and answer 503 when it does not respond); Swagger at `/swagger` only in the Development environment (`Program.cs:53-57`)
RabbitMQ	Dashboard at `http://localhost:15672`
Mailpit	Inbox at `http://localhost:8025`

With everything up, `lavarapido-infra/e2e` has three kinds of Node 22 scripts: the 19 end-to-end tests (`node --test "e2e/*.test.js"`: cross-service booking flow, full flow of the

three roles, inspection report and access rules), the response time measurement (`node e2e/performance.js`) and the check of the web at 360 px wide (`node e2e/width-360.js`, after `npm install --prefix e2e`). They need `MESSAGING_ENABLED=true` and `INTERNAL_API_KEY` in the `.env`. Since October 24, 2026 `e2e/remote` also has a battery of 44 cases run against the public server through the API (`node e2e/remote/run.js` or the `e2e.yml` workflow, by hand), with test accounts in `E2E_*` variables; it has not been run against Azure yet. The local run of the 19 end-to-end tests on October 27, 2026 passed 19 of 19, after the suite was adjusted to send the payment mode (`paymentMode`) when booking.

References

Angular. (n.d.). *Angular documentation*. Google. Retrieved October 9, 2026, from https://angular.dev/

Expo. (n.d.). *Expo documentation*. Retrieved October 9, 2026, from https://docs.expo.dev/

Microsoft. (n.d.). *.NET documentation*. Retrieved October 9, 2026, from https://learn.microsoft.com/dotnet/

React Native. (n.d.). *React Native documentation*. Meta Platforms. Retrieved October 9, 2026, from https://reactnative.dev/docs/getting-started

Spring. (n.d.). *Spring Boot reference documentation*. Broadcom. Retrieved October 9, 2026, from https://docs.spring.io/spring-boot/